Digital Twin-Driven Deep Reinforcement Learning for Energy-Aware CNC Milling Parameter Optimization

ME6324 – AI in Manufacturing

Course Project Report

Abhinand
Department of Mechanical Engineering
Indian Institute of Technology Madras
abhinand@iitm.ac.in

October 20, 2026

Git Revision: `87a4021` (main branch)

This report is generated from the project repository at `G:/My Drive/IITM/AI in Mfg`. All results are reproducible from the committed artifacts using the provided scripts.

Contents

1 Introduction **3**
- 1.1 Problem Statement . . . 3
- 1.2 Proposed Approach . . . 3
- 1.3 Contributions . . . 3

2 Related Work **4**

3 Digital Twin Physics **4**
- 3.1 Geometry and Kinematics . . . 5
- 3.2 Tool Wear (Extended Taylor Model) . . . 5
- 3.3 Surface Roughness (Empirical Regression) . . . 5
- 3.4 Energy Consumption . . . 5
- 3.5 Stochastic Tool Breakage . . . 5

4 Environment Design **6**
- 4.1 Registrations . . . 6
- 4.2 State Space . . . 6
- 4.3 Action Space . . . 6
- 4.4 Reward Function . . . 6
- 4.5 Episode Logic . . . 7
- 4.6 Domain Randomization . . . 7

5 Reinforcement Learning **7**
- 5.1 Algorithm . . . 7
- 5.2 Training Runs . . . 8
- 5.3 Monte Carlo Evaluation . . . 8

6 Classical Baselines **8**

7 Results **9**
- 7.1 Primary PPO Policy – Monte Carlo Sweep (100 Scenarios) . . . 9
- 7.2 Energy Weight Ablation – Pareto Frontier . . . 10
- 7.3 Conditioned Policy vs Ablation Sweep . . . 10
- 7.4 Baseline Comparison . . . 10
- 7.5 Transfer Experiment . . . 11

8 Interactive Digital Twin Workstation **11**

9 Software Engineering Practices **12**
- 9.1 Architecture . . . 12
- 9.2 Reproducibility . . . 12
- 9.3 Testing . . . 12
- 9.4 Code Quality . . . 13

10 Discussion **13**
- 10.1 What Worked . . . 13
- 10.2 Limitations . . . 13
- 10.3 Future Work . . . 14

11 Conclusion **14**

A Reproducibility Commands **15**

B Project Structure **16**

C Hyperparameter Sensitivity **17**

D Energy Model Derivation **17**

1 Introduction

1.1 Problem Statement

CNC face milling is a fundamental manufacturing operation where a rotating cutter removes material from a workpiece in multiple passes. The cutting parameters – spindle speed (n), feed per revolution (f_r), and axial depth of cut (a_p) – must be chosen to balance four competing objectives:

1. **Productivity:** Maximize material removal rate (MRR $= v_c \cdot a_p \cdot f_r$)
2. **Energy Efficiency:** Minimize total electrical energy consumption (E_{total}), accounting for both machine idle power and cutting power.
3. **Surface Quality Assurance:** Strictly adhere to finished workpiece roughness requirements ($R_a \leq 1.6$ μm).
4. **Tool Wear and Breakage Avoidance:** Limit cumulative flank wear ($V_B \leq 0.3$ mm) and mitigate catastrophic tool failure risks.

Traditional approaches use fixed parameters from handbooks or static optimization (e.g., NSGA-II) that pick one parameter set for the entire batch. These cannot adapt to the *state* of the process – tool wear accumulates, workpiece geometry changes, and material properties vary between batches.

1.2 Proposed Approach

This project develops a **Digital Twin** of the milling process – a high-fidelity physics-based simulator grounded in empirical manufacturing literature – and trains a **Deep Reinforcement Learning (DRL)** agent to dynamically optimize cutting parameters pass-by-pass. The agent observes the current state (flank wear, remaining stock, previous parameters) and selects the next pass parameters to maximize a reward that balances throughput and energy while respecting constraints.

Key innovations:

- **Speed-dependent specific cutting energy** $k_c(v_c) = k_{c0}(v_c/v_{\text{ref}})^m$ creates a genuine trade-off: faster cutting increases energy per unit volume but reduces idle energy.
- **Domain randomization** during training: material/tool coefficients, workpiece geometry, and initial wear are sampled per episode from literature-calibrated ranges.
- **Conditioned policy:** A single PPO policy conditioned on the energy weight w_2 and material context covers the entire Pareto frontier.
- **Transfer experiment:** Measures how much a policy trained on one material knows about a harder, faster-wearing material.
- **Receding-horizon controller baseline:** A strong classical benchmark that re-solves at every pass.

1.3 Contributions

1. A Gymnasium-compliant milling environment with explicit physics, energy-aware reward, and domain randomization.
2. A trained PPO policy evaluated via 100-scenario Monte Carlo sweep with 95% confidence intervals.

3. Energy-weight ablation study (7 weights × 3 seeds) mapping the empirical Pareto frontier.
4. Comparison against 5 classical baselines (Constant-High, Constant-Safe, Rule-Based, NSGA-II Static, Receding-Horizon).
5. Transfer experiment quantifying cross-material adaptation, which finds that fine tuning does not provide it and that training across a distribution of materials does.
6. Interactive Streamlit workstation with 2D stock depletion, micro-topography, G-code generation, and multi-controller comparison.
7. Full reproducibility: all results regenerated from committed artifacts by `agent/summarize_results.py`.

2 Related Work

This project positions itself against three key papers in the literature:

Table 1: Position against the three papers held in `docs/papers/`

Capability	Mo et al. (2025) *J. Manuf. Proc. 156*	Wang et al. (2025a) *J. Manuf. Sys. 78*	**This Work**
Dynamic parameter optimization	✓	✓	✓
Tool wear in the state	✓	✓	✓
Surface roughness constraint	✓	✓	✓
Energy as an objective	✗	✗	✓
Speed-dependent cutting energy	✗	✗	✓
Domain randomization	✗	✗	✓
Conditioned policy (single → frontier)	✗	✗	✓
Transfer experiment across materials	✗	✗	✓
Receding-horizon baseline	✗	✗	✓
Monte Carlo robustness (100 scenarios)	✗	✗	✓
Interactive digital twin workstation	✗	✗	✓

Both reference papers treat energy as related work rather than as an optimization target: in each of them energy appears in the discussion of prior art, not among the objectives. The third paper in `docs/papers/` (Wang et al., 2025b) solves the discrete step sequencing problem, so continuous cutting parameters are outside its scope. The speed-dependent cutting energy model (Section 3) is the key enabler here: without it the energy of a part is fixed by the volume removed, both objectives point the same way, and no weight on the energy term can open a trade-off. The conditioned policy reduces the ablation cost from 21 training runs to one. The receding-horizon controller provides a fair classical benchmark that uses the same physics and re-optimizes at every pass.

3 Digital Twin Physics

All equations are implemented in `config/physics.py` and used by the environment, baselines, simulator, and evaluation scripts – a single source of truth.

3.1 Geometry and Kinematics

Cutter diameter $D = 50$ mm, cutting length $L_{\text{cut}} = 100$ mm, total machining allowance $\kappa_{\text{total}} = 5.0$ mm.

$$v_c = \frac{\pi D n}{1000} \qquad [\text{m/min}] \tag{1}$$

$$\text{MRR} = v_c \cdot a_p \cdot f_r \qquad [\text{mm}^3/\text{min}] \tag{2}$$

$$t_{\text{cut}} = \frac{L_{\text{cut}}}{n f_r} \qquad [\text{min}] \tag{3}$$

$$N_{\text{passes}} = \left\lceil \frac{\kappa_{\text{total}}}{a_p} \right\rceil \qquad [\text{integer}] \tag{4}$$

3.2 Tool Wear (Extended Taylor Model)

Flank wear rate per pass:

$$\Delta V_B = C_{\text{wear}} \cdot v_c^{\alpha} \cdot f_r^{\beta} \cdot a_p^{\gamma_{\text{wear}}} \cdot t_{\text{cut}} \tag{5}$$

with calibrated coefficients $C_{\text{wear}} = 1.1 \times 10^{-4}$, $\alpha = 0.3$, $\beta = 0.2$, $\gamma_{\text{wear}} = 0.15$. Cumulative wear $V_{B,k} = \sum_{i=1}^{k} \Delta V_{B,i}$.

3.3 Surface Roughness (Empirical Regression)

$$R_a = k_1 f_r^2 + k_2 V_B + \frac{k_3}{v_c} \tag{6}$$

with $k_1 = 12.5$, $k_2 = 0.8$, $k_3 = 0.5$. The finished surface (last pass) must satisfy $R_a \le R_{a,\max} = 1.6$ μm.

3.4 Energy Consumption

Specific cutting energy grows with cutting speed:

$$k_c(v_c) = k_{c0} \left(\frac{v_c}{v_{\text{ref}}} \right)^{m}, \quad P_{\text{cut}} = k_c(v_c) \cdot \text{MRR}, \quad P_{\text{total}} = P_{\text{idle}} + P_{\text{cut}} \tag{7}$$

with $P_{\text{idle}} = 500$ W, $k_{c0} = 2.5$ W $\cdot$ min/mm^3, $m = 0.6$, v_{ref} from the reference cutting condition. Energy per pass: $E_{\text{pass}} = P_{\text{total}} \cdot t_{\text{cut}}$. Total energy per part: $E_{\text{part}} = \sum_{k=1}^{N_{\text{passes}}} E_{\text{pass},k}$.

Proposition 1 (Interior Minimum)**.** *With $m > 0$, the energy per part $E_{part}(v_c)$ has an interior minimum in v_c. The cutting term rises with v_c while the idle term falls, creating a genuine trade-off that the energy weight w_2 can select.*

This is validated by test `test_part_energy_has_an_interior_minimum_in_speed`.

3.5 Stochastic Tool Breakage

Probability of catastrophic breakage per pass, applied only once the flank wear passes a threshold:

$$p_{\text{break}} = 0.3 \cdot \underbrace{\frac{V_B - 0.15}{V_{B,\max} - 0.15}}_{\text{wear factor}} \cdot \underbrace{\left(\frac{\text{MRR}}{\text{MRR}_{\max}} \right)^2}_{\text{load factor}} \qquad (V_B > 0.15 \text{ mm, else } 0) \tag{8}$$

The draw comes from the environment's own generator, which `reset(seed=...)` seeds, so a scenario replayed with the same seed breaks in the same place. If triggered, the episode terminates with a -100 reward.

4 Environment Design

4.1 Registrations

Two Gymnasium environments are registered in `env/__init__.py`:

1. `MillingDTEnv-v1`: Base environment, fixed scenario per episode. Observation $\in \mathbb{R}^8$.
2. `MillingDTEnv-w2-v1`: Conditioned environment. Energy weight w_2 and material context appended to observation $\in \mathbb{R}^{11}$. Both $w_2 \sim \mathcal{U}[0,1]$ and scenario are drawn per `reset()`.

4.2 State Space

Table 2: Base observation (8 dimensions), every feature mapped onto $[0,1]$

Index	Feature	Normalization
0	Flank wear V_B	tanh squash over $[0, V_{B,\max}]$
1	Remaining stock κ_{rem}	tanh squash over $[0, \kappa_{\text{total}}]$
2	Previous spindle speed n	$(n - n_{\min})/(n_{\max} - n_{\min})$
3	Previous feed f_r	$(f_r - f_{\min})/(f_{\max} - f_{\min})$
4	Previous depth a_p	$(a_p - a_{p,\min})/(a_{p,\max} - a_{p,\min})$
5	Previous MRR	$\text{MRR}/\text{MRR}_{\max}$, clipped
6	Previous roughness R_a	tanh squash over $[0, R_{a,\max}]$
7	Tool broken	1 if broken, else 0
8	Energy weight w_2	$w_2 \in [0,1]$ (conditioned only)
9	Wear coefficient C_{wear}	over the scenario band $[0.6, 1.4]$ of nominal (conditioned only)
10	Roughness coefficient k_1	over the scenario band $[0.8, 1.2]$ of nominal (conditioned only)

Every feature is normalized against the *scenario's own* limits, so the observation is scale free across geometries and materials. The tanh squashing keeps the gradient magnitude when a constraint is violated, instead of clipping the value to 1.

4.3 Action Space

Continuous 3D box $\mathcal{A} = [-1,1]^3$, mapped to physical parameters:

$$n = n_{\min} + \frac{a_1 + 1}{2}(n_{\max} - n_{\min}) \tag{9}$$

$$f_r = f_{\min} + \frac{a_2 + 1}{2}(f_{\max} - f_{\min}) \tag{10}$$

$$a_p = a_{p,\min} + \frac{a_3 + 1}{2}(a_{p,\max} - a_{p,\min}) \tag{11}$$

The policy output is unbounded; the environment clips to $[-1,1]$ before mapping, guaranteeing the machine envelope is never violated.

4.4 Reward Function

The per-step reward is:

$$r = \underbrace{5\left(w_1 \frac{\text{MRR}}{\text{MRR}_{\max}} - w_2 \frac{E_{\text{pass}}}{E_{\text{ref}}}\right)}_{\text{Objective}} + \underbrace{2\,\Delta\left(\frac{\kappa_{\text{removed}}}{\kappa_{\text{total}}}\right)}_{\text{Progress}} - \underbrace{50\mathbb{1}[V_B > V_{B,\text{safe}}]}_{\text{Wear penalty}} - \underbrace{100\mathbb{1}[\text{broken}]}_{\text{Breakage}} - \underbrace{50\mathbb{1}[\text{finishing pass} \wedge R_a}_{\text{Roughness penalt}} \tag{12}$$

plus, on a part finished inside spec, a +200 completion bonus and a Pareto margin bonus of at most +5:

$$\mathrm{HV} = \max\Big(0, 1 - \tfrac{T_{\text{total}}}{T_{\text{ref}}}\Big) \cdot \max\Big(0, 1 - \tfrac{R_a}{R_{a,\text{safe}}}\Big) \cdot 5 \tag{13}$$

The objective term carries a scale factor of 5 so that it competes with the shaping terms. Without it the progress bonus and the constraint penalties decide the policy on their own, and the energy weight w_2 has no visible effect at all: the ablation produced 21 identical policies. The constraint penalties and the completion bonus are an order of magnitude larger on purpose, so that breaking a constraint is never worth a throughput or energy gain. $R_{a,\text{safe}} = 0.85\, R_{a,\max}$, $w_1 + w_2 = 1$, and E_{ref} is the energy of a pass under the reference condition, recomputed for every scenario from that scenario's diameter, cutting length and energy coefficients.

The reward is validated by `test_reward_matches_an_independent_calculation`, which recomputes it from `config/physics.py`, independently of the code that assembles it, and asserts agreement to 10^{-9}.

4.5 Episode Logic

- `terminated=True`: Part finished ($\kappa_{\text{rem}} \leq 0$), tool broken, or wear exceeded.
- `truncated=True`: Maximum steps (200) reached.
- Constraint violations on the *finished surface* incur the roughness penalty only at the final pass; intermediate passes are penalized via the safety margin $R_{a,\text{safe}}$.

4.6 Domain Randomization

When `randomize=True`, each `reset()` samples a new scenario from `config/scenarios.py`:

- Workpiece: $\kappa_{\text{total}} \in [2, 15]$ mm, $D \in [40, 150]$ mm, $L_{\text{cut}} \in [50, 200]$ mm
- Material/Tool: C_{wear} at $[0.6, 1.4]\times$ nominal, k_1, k_2, k_3 each at $[0.8, 1.2]\times$ nominal
- Initial wear: $V_{B,0} \sim \mathcal{U}[0, 0.8]\, V_{B,\max}$

The same tables are used for Monte Carlo evaluation, so the policy is measured on the distribution it was trained on. Measured on the trained policy, the observations the policy receives on the two sets agree to 0.041 in the mean of the normalized features, against a standard deviation of 1.

5 Reinforcement Learning

5.1 Algorithm

Proximal Policy Optimization (PPO) with MLP policy, implemented via Stable-Baselines3.

Table 3: PPO hyperparameters

Parameter	**Value**
Policy	MlpPolicy (2 hidden layers, 256 units each)
Learning rate	3×10^{-4} (linear schedule to 0)
n_steps	2048
Batch size	64
n_epochs	10
γ	0.99
GAE λ	0.95
Clip range	0.2
Entropy coefficient	0.02
Value function coefficient	0.5
Max grad norm	0.5
VecNormalize	`norm_obs=True, norm_reward=True`
Environments	4 parallel (`SubprocVecEnv`)
Device	CPU (2.5× faster than GPU for this MLP)

Training uses a custom `ProgressQueueCallback` that sends a heartbeat every 1000 steps to the supervisor (`run_experiments.py`); a dropped heartbeat triggers worker termination.

5.2 Training Runs

1. **Primary PPO:** 1M steps, seed 42, `randomize=True`. Model at `models/PPO_seed42/`.
2. **Conditioned PPO:** 1M steps, seed 42, on `MillingDTEnv-w2-v1`. Model at `models/PPO_conditioned_se`
3. **Ablation grid:** 7 weights $w_2 \in \{0.0, 0.2, 0.4, 0.5, 0.6, 0.8, 1.0\}\times$ 3 seeds $\{42, 43, 44\}$, 150k steps each. Trained in parallel via `run_experiments.py`.

5.3 Monte Carlo Evaluation

`agent/evaluate.py` runs 100 randomized scenarios (fixed seed 42) per policy. Each scenario is a full machining episode. Reports mean ± std and 95% CI (t-distribution) for:

- MRR, Energy/part, Cycle time
- Max R_a, Max V_B
- Constraint violation rates (worst pass vs finished surface)
- Tool breakage rate, Part completion rate

6 Classical Baselines

Five non-RL baselines implemented in `baselines/run_baselines.py`:

1. **Constant-High:** $n = 8000, f_r = 0.35, a_p = 2.5$ (upper bounds).
2. **Constant-Safe:** $n = 2000, f_r = 0.05, a_p = 0.1$ (lower bounds).
3. **Rule-Based Degradation:** Feed at maximum, spindle speed reduced by 20% every second pass, as a shop-floor heuristic for tool degradation.

4. **NSGA-II Static:** Multi-objective optimization (pymoo) picking *one* parameter set for the whole part. Decision variables $[n, f_r, a_p]$. Objectives: maximize MRR, minimize Energy. Constraints: $R_{a,\text{final}} \leq R_{a,\max}$, $V_{B,\text{final}} \leq V_{B,\max}$.

5. **Receding-Horizon Controller:** Vectorized grid search over (n, f_r, a_p) at *every pass*, simulating the remaining passes with the current V_B and κ_{rem}. Keeps feasible candidates, picks the best by the same scalarization $w_1\text{MRR} - w_2 E$. This is the strongest classical baseline.

All baselines use the *same physics* (`config/physics.py`) and are evaluated on the same 50 episodes of the nominal machining condition.

7 Results

All tables below are regenerated from committed CSVs by `python agent/summarize_results.py`.

7.1 Primary PPO Policy – Monte Carlo Sweep (100 Scenarios)

Table 4: Primary PPO Monte Carlo results

Metric	**Mean**	**Std**	**95% CI**
Material removal rate (mm^3/min)	1239.26	621.25	[1115.99, 1362.53]
Energy per part (kJ)	68.20	48.68	[58.54, 77.87]
Cycle time per part (s)	19.38	14.13	[16.58, 22.18]
Max surface roughness (μm)	1.475	0.235	[1.428, 1.521]
Max flank wear (mm)	0.1244	0.0679	[0.1109, 0.1379]
Constraint violations			
Roughness violations, worst pass	39.0%		[29.4%, 48.6%]
Roughness violations, finished surface	0.0%		[0.0%, 0.0%]
Roughness > safety limit, worst pass	61.0%		[51.4%, 70.6%]
Roughness > safety limit, finished surface	0.0%		[0.0%, 0.0%]
Flank wear violations ($V_B > 0.3$ mm)	0.0%		[0.0%, 0.0%]
Tool breakage	1.0%		[0.0%, 3.0%]
Parts completed within spec	99.0%		
Scenarios	100		
Mean operating point	7241 RPM, 0.254 mm/r, 2.11 mm, 4.0 passes		

Key observations:

- **Finished surface always within spec:** 0% roughness violations on the final pass.
- **Wear never exceeded:** 0% flank wear violations.
- **Intermediate roughness:** 61% of passes exceed the safety margin ($0.85 \times 1.6 = 1.36$ μm), but the policy recovers by the finish pass.
- **High variance:** Std of MRR is 50% of mean – domain randomization produces diverse operating points.

7.2 Energy Weight Ablation – Pareto Frontier

Table 5: Ablation study: mean ± std over 3 seeds, 50 scenarios each

w_2	MRR (mm^3/min)	Energy (kJ)	Cycle (s)	n (RPM)	f_r (mm/r)	Frontier
0.0	999.9 ± 31.3	82.8 ± 0.7	45.8 ± 2.0	6974	0.177	dominated
0.2	990.5 ± 137.3	76.3 ± 2.2	31.2 ± 5.8	7307	0.186	dominated
0.4	1069.0 ± 40.0	71.5 ± 0.4	22.9 ± 1.0	7205	0.212	✓
0.5	1026.2 ± 56.9	71.0 ± 0.7	23.2 ± 0.5	7115	0.208	✓
0.6	1008.4 ± 22.6	68.3 ± 0.8	21.6 ± 1.1	6775	0.223	✓
0.8	888.3 ± 57.6	65.8 ± 1.4	23.1 ± 0.9	6307	0.222	✓
1.0	876.6 ± 43.2	65.4 ± 1.7	22.7 ± 1.2	6321	0.220	✓

Proposition 2 (Frontier Shape)**.** *Across the Pareto frontier ($w_2 \in [0.4, 1.0]$):* MRR *falls 18.0% (1069 → 877 mm^3/min) while energy falls 8.5% (71.5 → 65.4 kJ). The trade-off is asymmetric – large* MRR *sacrifice for modest energy gain – because the speed-dependent cutting energy makes high speeds energetically expensive.*

The dominated points ($w_2 = 0.0, 0.2$) have *higher* energy than $w_2 = 0.4$ because they push speed too high, increasing $k_c(v_c)$ without reducing idle energy enough to compensate.

7.3 Conditioned Policy vs Ablation Sweep

The conditioned policy (`MillingDTEnv-w2-v1`) is evaluated by sweeping w_2 at runtime and compared against the ablation policies (one per weight).

Table 6: Conditioned policy runtime sweep against the per-weight sweep

w_2	Conditioned, 1M steps MRR (mm^3/min)	Per-weight, 150k steps MRR (mm^3/min)	Energy gap conditioned vs sweep
0.0	1213.6	999.9	−15.9%
0.2	1215.3	990.5	−9.5%
0.4	1205.1	1069.0	−4.4%
0.5	1191.0	1026.2	−4.2%
0.6	1169.3	1008.4	−1.0%
0.8	1130.8	888.3	+2.0%
1.0	1096.0	876.6	+2.6%

A single conditioned policy of 1,000,000 steps covers the whole weight range and beats the 21 weight-specific policies of 150,000 steps each, which is 3,150,000 steps in total: 13% to 27% more material removal rate for at most 3% more energy. At a matched budget of 150,000 steps the two frontiers cross: the per-weight policies reach the throughput-heavy corner and the conditioned policy the energy-heavy one. The energy weight stays a runtime query rather than a retrain.

7.4 Baseline Comparison

All six policies are evaluated on the same 50 episodes of the nominal machining condition, so the numbers are comparable. The reward column is the decisive one, because it carries the constraint penalties.

Table 7: Baseline comparison, 50 episodes of the nominal condition

Policy	MRR **(mm^3/min)**	**Energy (kJ)**	**Cycle (s)**	**Mean reward**
PPO (primary)	877.7	18.5	5.7	**+204.1**
Receding-Horizon	**981.7**	**18.0**	**4.8**	**+203.5**
NSGA-II Static	732.9	18.8	6.4	−46.4
Rule-Based	719.8	18.7	6.6	−46.5
Constant-High	1099.6	17.8	4.3	−46.1 (violates)
Constant-Safe	1.6	1506.8	3000.0	−139.3

- **The four static policies end outside the roughness safety limit**, which the reward penalizes with −50. The receding-horizon controller is the only classical policy that stays inside it, and it reaches +203.5 against −46 for the rest.
- **The learned policy and the controller are level on the reward** (+204.1 against +203.5), and the controller is ahead on the removal rate, the energy and the cycle time.
- **This digital twin does not yet show a reinforcement learning advantage.** The next experiment is to initialize the policy from the controller and fine-tune from there.

7.5 Transfer Experiment

Target material: harder alloy ($k_1 \times 1.25$, $k_2 \times 1.30$) with faster-wearing tool ($C_{\text{wear}} \times 2.0$). Source policy never saw this material.

Table 8: Transfer experiment on the target material

Arm	**Mean reward**	**Roughness violations**	**Parts completed**
Source policy, zero-shot	183.0	100.0%	93.3%
Source policy, fine-tuned (50k steps)	182.9	100.0%	93.3%
Target from scratch (50k steps)	196.7	0.0%	100.0%

Fine tuning is the wrong tool for a material change. The source policy violates the roughness limit in every scenario of the harder material, and 50,000 steps of fine tuning move the reward from 183.0 to 182.9 and remove no violation at all. A policy trained on the harder material from scratch, for the same budget, reaches 196.7 with no violations. The remedy is the one this project adopted: train across a distribution of materials, so that no adaptation step is needed at all.

8 Interactive Digital Twin Workstation

The Streamlit application (`app.py`) provides an industrial-grade interface:

1. **2D Cross-Sectional Stock Depletion:** Real-time visualization of cylindrical billet, pass-by-pass cutout, remaining allowance κ, depth per pass a_p, and tool engagement.
2. **Micro-Topography Generator:** Feed-mark cusp profile $y_{\text{cusp}}(x) \approx x^2/(8r_\epsilon)$+stochastic perturbation with ISO 1302 centerline average R_a overlay and 1.6 μm ceiling.
3. **ISO 6983 G-Code Synthesizer:** Emits Fanuc/Siemens compliant G-code for the exact PPO parameter sequence.

4. **Multi-Controller Comparison:** Side-by-side PPO vs NSGA-II vs Rule-Based vs Constant-High with telemetry overlay.

5. **Cached Policy Loading:** `@st.cache_resource` ensures model loads once per session.

Launch: `streamlit run app.py`

9 Software Engineering Practices

9.1 Architecture

- `config/`: Physics constants, equations, scenario distributions, paths (single source of truth)
- `env/`: Gymnasium environment, two registrations
- `agent/`: Training, evaluation, ablation, transfer, pipeline orchestration
- `baselines/`: 5 classical baselines + receding-horizon controller
- `sim/`: Simulation engine for Streamlit (stock depletion, micro-topography, G-code)
- `viz/`: Plotting utilities, training curves, MC distributions, baseline comparison
- `utils/`: Seeding, reproducibility

9.2 Reproducibility

- Global seeding via `utils/seeding.py` (Python, NumPy, PyTorch, CUDA deterministic)
- `config/scenarios.py` shared by training and Monte Carlo – no drift
- `agent/summarize_results.py` rebuilds `RESULTS.md` from committed CSVs
- All random draws (scenarios, weights, seeds) are deterministic given the seed

9.3 Testing

26 tests in `tests/` covering:

- Environment: action clipping, truncation, soft norm, physics constants, overrides, info dict, energy norm, Gym checker, randomization, pinned values, scenario scales, cutting energy speed dependence, part energy interior minimum, shared physics, shared scenario tables
- Reward: golden-value test (independent recomputation), finishing-pass penalty
- Integration: training→evaluation roundtrip, conditioned observation context, baseline artifact
- Progress callback: heartbeat delivery, failure reporting, interval respect

9.4 Code Quality

- Centralized constants (`config/physics_constants.py`) with literature citations
- Centralized paths (`config/paths.py`) – no hardcoded strings
- Environment IDs exported from `env/__init__.py`
- No unused imports (AST-verified)
- Conventional commits, clean git history

10 Discussion

10.1 What Worked

1. The speed-dependent cutting energy model ($k_c \propto v_c^m$) is essential – without it, the energy/throughput trade-off vanishes and 21 ablation policies come out identical.
2. Domain randomization + shared scenario tables: the policy generalizes to unseen scenarios (99% part completion, 0% finished-surface violations).
3. The conditioned policy reduces the ablation cost from 21 training runs to one, and beats them at a third of the total step budget.
4. The receding-horizon controller is a strong, fair baseline using the same physics. It also exposed that the earlier baseline comparison was hollow.
5. The test suite catches physics regressions (energy interior minimum, cutting energy speed dependence) and reward regressions (golden-value test).

10.2 Limitations

1. The learned policy does not outperform the receding-horizon controller on the reward (Table 7), and the controller is ahead on the physical metrics. The controller re-solves with perfect physics; the policy has to learn the dynamics from experience.
2. The energy weight ablation uses 150k steps per weight (vs 1M for the primary policy) – the frontier may shift with more training.
3. The reward charges the energy of a pass, while the reported metric is the energy of a part. A part cut in many shallow passes draws less power per pass but spends more time on the idle machine, so the two can move in opposite directions. Both are reported.
4. Stochastic tool breakage (1% rate) introduces variance; the reward penalty could be shaped more carefully.
5. The micro-topography model is a simplified feed-mark cusp; real chatter and vibration are not modeled.
6. The energy model is the modelling choice of this work, not a result taken from the reference papers, and it is calibrated against a single reference condition.

10.3 Future Work

1. **Controller-initialized RL:** Initialize PPO from the receding-horizon controller's trajectories (behavior cloning) then fine-tune with RL.
2. **Sim-to-real:** Calibrate physics coefficients on real machine data; add sensor noise and delay to the environment.
3. **Multi-objective RL:** Replace scalarization with Pareto-conditioned networks or hypervolume maximization.
4. **Online adaptation:** Deploy the conditioned policy with a runtime optimizer that adjusts w_2 based on energy budget.
5. **Full 3D Digital Twin:** Extend the 2D cross-section to 3D toolpath visualization with collision checking.

11 Conclusion

This project demonstrates a complete pipeline for DRL-based CNC milling optimization:

1. A physics-grounded Digital Twin with speed-dependent cutting energy, tool wear, surface roughness, and stochastic breakage.
2. A Gymnasium environment with domain randomization, conditioned observations, and an energy-aware reward whose objective terms are scaled to compete with the shaping terms.
3. PPO policies trained with proper experimental rigor: Monte Carlo evaluation with confidence intervals, ablation study mapping the empirical Pareto frontier, conditioned policy covering the frontier in one run.
4. Strong classical baselines including a receding-horizon controller that re-optimizes at every pass.
5. A transfer experiment that measures cross-material adaptation, and finds that fine tuning does not provide it.
6. An interactive workstation for visualization, G-code generation, and multi-controller comparison.
7. Full reproducibility: every result table is generated from committed artifacts by a single script.

The key scientific finding is that **the speed-dependent specific cutting energy creates a genuine energy/throughput trade-off**, and a DRL agent can learn to navigate it pass-by-pass while respecting surface quality and tool life constraints. Across the measured frontier the removal rate falls 18% while the energy of the part falls 8.5%, and the spindle speed falls with the energy weight, which is the mechanism the model predicts. The conditioned policy architecture makes the whole trade-off accessible at runtime with a single model.

All code, data, and results are available in the repository at `G:/My Drive/IITM/AI in Mfg`.

References

References

[1] X. Mo, M. Xia, S. Mei, X. Hu, "Deep reinforcement learning-based optimization for machining process: Cutting parameters dynamic optimization considering tool condition degradation," *Journal of Manufacturing Processes* 156 (2025) 1006–1031.

[2] P. Wang, Y. Cui, H. Tao, X. Xu, S. Yang, "Machining parameter optimization for a batch milling system using multi-task deep reinforcement learning," *Journal of Manufacturing Systems* 78 (2025) 124–152, doi: 10.1016/j.jmsy.2024.11.013.

[3] P. Wang, Y. You, W. Liu, D. Ding, F. Xu, "Knowledge-guided deep reinforcement learning framework for feature machining step sequencing," *International Journal of Advanced Manufacturing Technology* 140 (2025) 5617–5636, doi: 10.1007/s00170-025-16606-4.

A Reproducibility Commands

All commands run from the project root `G:/My Drive/IITM/AI in Mfg`:

Listing 1: Key reproducibility commands

```
# Run all tests
pytest tests/ -v

# Train primary PPO (1M steps)
python agent/train_rl.py --algo PPO --steps 1000000 --seed 42 --
    randomize

# Train conditioned policy (1M steps)
python agent/train_rl_conditioned.py --steps 1000000 --seed 42

# Monte Carlo evaluation
python agent/evaluate.py

# Ablation study (7 weights x 3 seeds, 150k steps each)
python agent/train_rl_ablation.py --steps 150000 --scenarios 50 --slots
    7

# Classical baselines
python baselines/run_baselines.py

# Transfer experiment
python agent/transfer_experiment.py

# Regenerate all plots
python viz/plot_eval.py
python viz/plot_results.py
python viz/plot_baselines_comparison.py

# Regenerate RESULTS.md from artifacts
python agent/summarize_results.py

# Launch interactive workstation
streamlit run app.py

# Full pipeline (sequential)
```

```
python agent/run_pipeline.py --steps 1000000 --ablation_steps 100000 --
    scenarios 50 --slots 7
```

B Project Structure

Listing 2: Project directory tree

```
G:\My Drive\IITM\AI in Mfg
|-- agent/
|   |-- train_rl.py                 # Primary PPO training
|   |-- train_rl_conditioned.py     # Conditioned policy training
|   |-- train_rl_ablation.py        # Energy weight ablation
|   |-- evaluate.py                 # Monte Carlo sweep
|   |-- evaluate_conditioned.py     # Conditioned policy sweep
|   |-- transfer_experiment.py      # Cross-material transfer
|   |-- run_experiments.py          # Parallel training supervisor
|   |-- run_pipeline.py             # Full pipeline orchestrator
|   |-- summarize_results.py        # Regenerates RESULTS.md
|   `-- __init__.py
|-- baselines/
|   |-- run_baselines.py            # 5 classical baselines
|   |-- receding_horizon.py         # Receding-horizon controller
|   `-- __init__.py
|-- config/
|   |-- physics_constants.py        # All constants with citations
|   |-- physics.py                  # Shared physics equations
|   |-- scenarios.py                # Randomized scenario sampling
|   |-- paths.py                    # Centralized paths
|   `-- __init__.py
|-- env/
|   |-- milling_env.py              # Gymnasium environment
|   `-- __init__.py                 # Exports ENV_ID,
    CONDITIONED_ENV_ID
|-- sim/
|   |-- machining_sim.py            # Stock depletion, micro-topography
    , G-code
|   `-- __init__.py
|-- viz/
|   |-- plotting_utils.py           # Shared plotting functions
|   |-- plot_eval.py                # Monte Carlo distributions
|   |-- plot_results.py             # Training curves
|   |-- plot_baselines_comparison.py # Baseline comparison charts
|   `-- __init__.py
|-- utils/
|   `-- seeding.py                  # Global deterministic seeding
|-- tests/
|   |-- test_env.py                 # 19 environment tests
|   |-- test_training.py            # 2 training tests
|   |-- test_progress_callback.py   # 3 callback tests
|   `-- test_integration.py         # 3 integration tests
|-- results/                        # Committed artifacts (CSVs, plots)
|-- models/                         # Trained models (gitignored,
    regenerated)
|-- app.py                          # Streamlit workstation
|-- AUDIT_REPORT.md                 # Code audit with dispositions
|-- PROJECT.md                      # Consolidated documentation
|-- README.md                       # Project overview
```

```
‘-- report/
    ‘-- report.tex                    # This file
```

C Hyperparameter Sensitivity

The PPO hyperparameters in Table 3 are the Stable-Baselines3 defaults, with three exceptions: the network is widened to 2×256 units, the entropy coefficient is raised to 0.02 to keep the policy from collapsing to deterministic actions early, and the training runs on the CPU with one thread per process so that several policies can train in parallel on one machine. Measured on this machine, PPO on this MLP runs at 812 steps/s on the CPU against 318 steps/s on the GPU, so the GPU is not used.

The entropy coefficient matters most: with too little of it the policy settles into a deterministic action early and stops exploring the envelope.

D Energy Model Derivation

The specific cutting energy $k_c(v_c)$ models the empirical observation that the energy required to remove a unit volume of material increases with cutting speed due to:

1. Higher shear zone temperatures $\rightarrow$ increased flow stress
2. Increased strain rates $\rightarrow$ strain-rate hardening
3. Tool-chip interface friction changes

The power-law form $k_c(v_c) = k_{c0}(v_c/v_{\text{ref}})^m$ with $m \approx 0.6$ is a standard empirical form for the rise of the specific cutting energy with cutting speed, and it is the modelling choice of this work rather than a result taken from either reference paper: the reference papers optimize quality and efficiency, and treat energy only as related work. The reference condition v_{ref} is the cutting speed at the calibrated nominal parameters ($n = 5000$ RPM, $D = 50$ mm $\rightarrow v_{\text{ref}} \approx 785$ m/min). At this speed, $k_c = k_{c0} = 2.5$ W·min/mm^3.

Total power: $P_{\text{total}} = P_{\text{idle}} + k_c(v_c) \cdot \text{MRR}$. Energy per pass: $E_{\text{pass}} = P_{\text{total}} \cdot t_{\text{cut}}$. Substituting $t_{\text{cut}} = L_{\text{cut}}/(nf_r)$ and $\text{MRR} = v_c a_p f_r$:

$$E_{\text{pass}} = P_{\text{idle}} \frac{L_{\text{cut}}}{nf_r} + k_{c0} \left(\frac{v_c}{v_{\text{ref}}} \right)^m v_c a_p f_r \frac{L_{\text{cut}}}{nf_r} \tag{14}$$

$$= \underbrace{P_{\text{idle}} \frac{L_{\text{cut}}}{nf_r}}_{\text{Idle energy}} + \underbrace{k_{c0} \frac{L_{\text{cut}} a_p}{nf_r} \left(\frac{\pi D n}{1000 v_{\text{ref}}} \right)^m \frac{\pi D n}{1000}}_{\text{Cutting energy}} \tag{15}$$

The idle term $\propto 1/(nf_r)$ decreases with speed; the cutting term $\propto n^m$ increases with speed. Their sum has an interior minimum, which is the operating point selected by w_2.